

Mecanismo de atención (Query, Key, Value)

Módulo 1 · Fundamentos Ontológicos y Razonamiento de Máquinas

Como Q, K y V calculan relevancia contextual, por que multi-head attention captura multiples relaciones y las implicaciones para diseno de prompts.

1. Introducción · 2. Qué es · 3. Cómo funciona · 4. Ejemplos reales
5. Errores comunes · 6. Aplicación práctica · 7. Relación con módulos · 8. Resumen ejecutivo

Guía de estudio detallada · +2.000 palabras

El mecanismo de atención: Query, Key y Value

El mecanismo de atención es la innovación técnica central del Transformer y el componente que más directamente explica sus capacidades. Permite que cada posición en la secuencia preste atención de forma diferenciada a todas las demás posiciones, calculando cuánta relevancia tiene cada parte del contexto para producir la representación actual. Entender cómo funciona matemáticamente es lo que permite entender por qué los LLMs pueden resolver referencias anafóricas complejas, mantener coherencia a lo largo de documentos largos, y razonar sobre relaciones entre elementos distantes en el texto.

El paper de Vaswani et al. describe la atención como "una función que mapea una query y un conjunto de pares clave-valor a un output, donde la query, las claves, los valores y el output son todos vectores". Esta descripción abstracta se concreta en una operación matricial elegante que, repetida en múltiples "cabezas" en paralelo, produce la capacidad de capturar diferentes tipos de relaciones simultáneamente.

La intuición de Q, K, V con una analogía concreta

La intuición más clara para el mecanismo QKV usa una analogía de búsqueda en una biblioteca. Imagina que cada libro tiene un título (Key) y un contenido (Value). Cuando buscas información, haces una consulta (Query). El mecanismo de atención calcula cuánto se parece tu consulta a cada título (similitud $Q \cdot K$), usa esas similitudes para ponderar los contenidos (multiplicar por V), y produce como resultado una combinación ponderada de los contenidos más relevantes para tu consulta.

En el contexto de los Transformers, esto ocurre de forma paralela para todos los tokens de la secuencia simultáneamente. Cada token actúa como consulta (pregunta "¿qué información del contexto es relevante para representarme?"), como clave (anuncia "esto es lo que represento") y como valor (contribuye "esto es lo que apporto si eres relevante para mí"). El resultado es que la representación de cada token absorbe información contextualizada de todos los demás tokens ponderada por relevancia.

Las matemáticas de la atención: paso a paso

Dado un conjunto de tokens representados como una matriz X (n tokens $\times$ d dimensiones de embedding), el mecanismo de atención produce tres matrices proyectadas: $Q = X \cdot W_Q$ (queries), $K = X \cdot W_K$ (keys), $V = X \cdot W_V$ (values), donde W_Q , W_K , W_V son matrices de pesos aprendidas durante el entrenamiento. La dimensión de Q , K , V es d_k (típicamente d/h donde h es el número de cabezas).

Los scores de atención se calculan como el producto punto de Q y K transpuesto: QK^T . Esto produce una matriz $n \times n$ donde cada entrada (i,j) mide cuánto el token i debe atender al token j . Este producto se escala dividiendo por $\sqrt{d_k}$ (para estabilizar los gradientes, porque valores

grandes de d_k producen productos punto muy grandes que llevan a softmax saturado). Se aplica softmax por filas para convertir los scores en pesos de atención que suman 1. Finalmente, el output es la multiplicación de estos pesos por V.

La atención enmascarada (masked self-attention) en los decoders añade una máscara que pone los scores de posiciones futuras a $-\infty$ antes del softmax, de forma que el token en posición i no puede atender a tokens en posiciones $j > i$. Esto garantiza que la generación sea causal: cada token se genera basándose solo en los tokens anteriores, no en los futuros.

Multi-head attention: perspectivas múltiples simultáneas

La multi-head attention ejecuta h instancias de la atención en paralelo, cada una con sus propias matrices W_Q , W_K , W_V (proyecciones a espacios de dimensión más pequeña). Los outputs de las h cabezas se concatenan y se proyectan de vuelta a la dimensión original. Esto permite que diferentes cabezas capturen diferentes tipos de relaciones: algunas cabezas capturan relaciones sintácticas (sujeto-verbo-objeto), otras relaciones semánticas, otras relaciones de coreferencia. GPT-2 small tiene 12 cabezas; modelos modernos tienen 64, 96 o más.

SECCIÓN 4 · EJEMPLOS REALES

- Resolución de coreferencia: en "María fue al mercado. Ella compró manzanas", la atención alta entre "Ella" y "María" es lo que permite al modelo saber de quién se habla. Esta relación de largo alcance es imposible en modelos sin atención.
- Comprensión de negación: "El proyecto NO cumple los requisitos" vs "El proyecto cumple los requisitos". La cabeza de atención que detecta "NO" y lo asocia con "cumple" es lo que permite al modelo entender la negación correctamente.
- KV Cache para inferencia eficiente: en la generación autoregresiva, los valores K y V de los tokens ya procesados no cambian. El KV cache almacena estos valores calculados para reutilizarlos, reduciendo el coste de generación en inferencia. Esta optimización es fundamental para hacer viables los despliegues de LLMs a escala.
- Flash Attention: una reimplementación del mecanismo de atención que opera directamente en SRAM (memoria rápida) en lugar de HBDM (memoria lenta de GPU), reduciendo el tiempo de entrenamiento hasta un 3-8x sin cambiar los resultados matemáticos.

SECCIÓN 5 · ERRORES Y MALENTENDIDOS COMUNES

Error 1: "La atención mide cuánto un token importa en general." La atención es relativa: mide cuánto token i importa para computar la representación del token j en este contexto específico. Una misma palabra puede tener distribuciones de atención completamente diferentes en diferentes contextos.

Error 2: "Los pesos de atención son interpretables directamente." La interpretación de pesos de atención es un área activa de investigación con debates abiertos. Aunque hay correlaciones interesantes (las cabezas que atienden a verbos para sujetos, etc.), los pesos de atención no son directamente interpretables como "importancia".

Error 3: "La atención causal es solo para decoders." Tiene implicaciones para el diseño de cualquier sistema de LLM generativo: el orden de la información en el prompt importa

porque los tokens anteriores pueden atender a los posteriores pero no al revés. Esto afecta a cómo se estructuran los prompts.

SECCIÓN 6 · APLICACIÓN PRÁCTICA

El conocimiento del mecanismo de atención tiene tres aplicaciones prácticas directas. Primera: diseño de prompts más efectivos. Entender que los tokens anteriores del contexto afectan las representaciones de los tokens posteriores, y que la posición importa (lost-in-the-middle), guía la estructuración de los prompts para maximizar la atención en la información más relevante.

Segunda: optimización de costes. El KV cache, Flash Attention y Grouped Query Attention son optimizaciones del mecanismo de atención que reducen costes de inferencia. Entender por qué existen permite evaluar inteligentemente las características de los frameworks de serving (vLLM, TGI) que las implementan.

Tercera: diagnóstico de fallos. Cuando un LLM "olvida" información de contextos largos o produce incoherencias en documentos extensos, el diagnóstico correcto es frecuentemente un problema de capacidad de atención, no de "inteligencia" del modelo. La solución puede ser reestructurar el contexto, usar RAG para reducir la longitud del contexto, o usar un modelo con ventana de contexto mayor.

SECCIÓN 7 · RELACIÓN CON OTROS MÓDULOS

- M1-T8 (Transformer): el mecanismo de atención es el componente central del Transformer, profundizado aquí.
- M6 (Ingeniería de Prompts): el diseño de prompts efectivos tiene su fundamento técnico en entender cómo funciona la atención.
- M8 (LLMOps): el KV cache, Flash Attention y las optimizaciones de serving son implementaciones de ingeniería del mecanismo de atención.

SECCIÓN 8 · RESUMEN EJECUTIVO

El mecanismo de atención QKV calcula para cada token una representación contextualizada que pondera las contribuciones de todos los demás tokens según su relevancia (scores $Q \cdot K^T / \sqrt{d_k}$, normalizados por softmax y aplicados sobre V). La multi-head attention ejecuta este mecanismo h veces en paralelo con diferentes proyecciones, capturando diferentes tipos de relaciones simultáneamente. El coste $O(n^2)$ es la limitación técnica central. Aplicaciones prácticas: diseño de prompts (posición importa), optimizaciones de inferencia (KV cache, Flash Attention) y diagnóstico de fallos en contextos largos.